

Functions in Python

A **function** in Python is a reusable block of code that performs a specific task. Functions help organize programs, reduce code repetition, and make code easier to understand, test, and maintain.

Syntax:

```
def function_name(parameters):  
    # Code to execute  
    return value
```

<i>def</i>	<i>A keyword used to define a function.</i>
<i>function_name</i>	<i>The name given to the function.</i>
<i>parameters</i>	<i>Values passed to the function (optional).</i>
<i>return</i>	<i>Used to send a result back from the function (optional).</i>

Types of Functions in Python

1. Built-in Functions

Built-in functions are pre-defined functions provided by Python. They can be used directly without creating them.

Examples:

```
print("Hello")
```

```
len("Python")
```

```
max(5, 10, 15)
```

2. User-defined Functions

User-defined functions are created by programmers to perform specific tasks.

Example:

```
def square(n):  
    return n * n  
print(square(5))
```

Output:

25

3. Lambda Functions

Lambda functions are small anonymous functions that contain only one expression.

Example:

```
square = lambda x: x * x  
print(square(4))
```

Output:

16

Creating Functions in Python

1. Function without Parameters

A function can be created without any input values.

Example:

```
def greet():  
    print("Hello, Welcome to Python!")  
  
greet()
```

Output:

```
Hello, Welcome to Python!
```

2. Function with Parameters

Parameters allow a function to accept input values.

Example:

```
def greet(name):  
    print("Hello,", name)  
  
greet("Alice")
```

Output:

```
Hello, Alice
```

3. Function with Return Value

A return statement sends a value back from the function.

Example:

```
def add(a, b):  
    return a + b  
  
result = add(10, 20)  
  
print(result)
```

Output:

30

Types of Function Arguments

1. Positional Arguments

Values are passed according to their position.

Example:

```
def info(name, age):  
    print(name, age)  
  
info("John", 25)
```

2. Keyword Arguments

Values are passed using parameter names.

Example:

```
def info(name, age):  
    print(name, age)  
  
info(age=25, name="John")
```

3. Default Arguments

A default value is assigned to a parameter if no value is provided.

Example:

```
def greet(name="Guest"):  
    print("Hello", name)  
  
greet()  
greet("Emma")
```

Output:

```
Hello Guest  
Hello Emma
```

4. Variable-Length Arguments

Variable-length arguments allow a function to accept multiple values.

Example:

```
def total(*numbers):  
    return sum(numbers)
```

```
print(total(1, 2, 3, 4))
```

Output:

```
10
```

Recursive Function Example

A recursive function is a function that calls itself to solve a problem.

Example: Factorial of a Number

```
def factorial(n):  
    if n == 0 or n == 1:  
        return 1  
    return n * factorial(n - 1)
```

```
print(factorial(5))
```

Output:

```
120
```